# Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026



## Contents

# Part I. Foundations

## 1. What is Deep Learning?

### 1.1. Demonstration

▹ Session 1, Slides 1-12

Before any theory, consider the following experiment. A large language model - specifically Claude - is prompted with a single sentence:

> *"Write me a PyTorch script that trains a neural network to classify CIFAR-10 images into 10 classes. Use a simple MLP architecture. Print the training loss each epoch and plot a few example predictions with their true and predicted labels at the end."*

The model produces a complete, runnable Python script. When executed, the script downloads the CIFAR-10 dataset - 60,000 colour images of $32 \times 32$ pixels across 10 classes - trains a neural network, and achieves reasonable classification accuracy. A second prompt requesting a CNN instead of an MLP produces a different script that achieves noticeably higher accuracy on the same task.

This demonstration works. It is genuinely easy. And therein lies the problem: *what just happened?*

The script contains weights, layers, a loss function, a training loop, gradient updates. None of these were explained. The goal of this course is to open that black box - not to prompt better, but to think deeper.

### 1.2. What is Deep Learning?

▹ Session 1: What is Deep Learning?

Artificial intelligence, machine learning, and deep learning are often used interchangeably in popular discourse, but they refer to distinct and nested concepts.

**Artificial intelligence** is the broadest term - it encompasses any technique that enables machines to exhibit behaviour we would consider intelligent, including rule-based expert systems, search algorithms, and learned models.

**Machine learning** is a subfield of AI in which systems learn from data rather than being programmed with explicit rules. Instead of writing down every decision, we provide examples and let the system discover the underlying patterns.

**Deep learning** is a subfield of machine learning that uses neural networks with many layers - so-called deep neural networks. The depth allows the network to learn increasingly abstract representations of the input data.

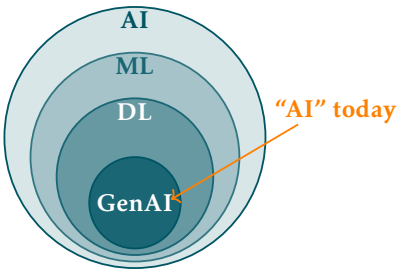


Figure 1.1: The relationship between AI, machine learning, deep learning, and generative AI.

**Generative AI** - encompassing large language models as well as image, video, and music generation systems - is a specific and currently prominent subset of deep learning. What the general public refers to as "AI" today is almost exclusively generative AI, which sits at the innermost level of this hierarchy.

**Definition 1** (Deep Learning)**.** Deep learning is a class of machine learning methods that use multi-layered neural networks to learn hierarchical representations directly from raw data.

This definition will sharpen as the course progresses. For now it is sufficient to hold it loosely - by the end of the course every word in it will have a concrete meaning.

## 1.3. Brief History

▹ Session 1: A Brief History of Deep Learning

Figure 1.2 shows the key milestones in the development of deep learning alongside the periods of reduced funding and interest known as AI winters. For a comprehensive overview of the field's history see LeCun, Bengio, and G. Hinton 2015 and Goodfellow, Bengio, and Courville 2016.

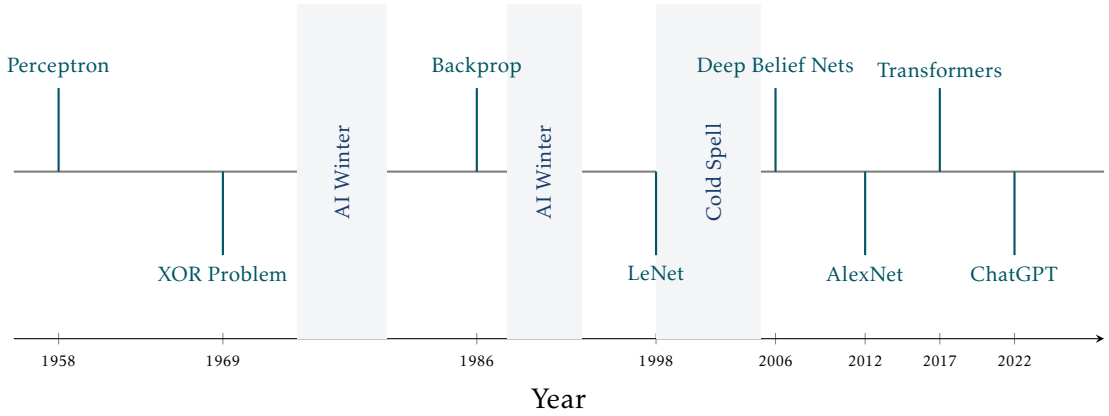


Figure 1.2: Key milestones in the history of deep learning. Shaded regions indicate AI winters and the cold spell of the early 2000s.

**1958 - Perceptron.** Frank Rosenblatt 1958 introduced the perceptron - the first trainable neural network. It could learn to classify linearly separable patterns by adjusting weighted connections between inputs and a single output unit. The result generated enormous excitement and equally enormous overpromising about the future of machine intelligence. Rosenblatt himself claimed the perceptron would eventually be able to walk, talk, see, write, reproduce itself, and be conscious of its existence - a level of optimism that would later contribute to the severity of the backlash.

The years following the perceptron saw genuine progress. The ADALINE model Widrow and Hoff 1960 introduced the least mean squares learning rule, which remains in use today. The concept of multilayer networks was understood theoretically, and researchers were actively exploring how to train them. There was a widespread belief that human-level artificial intelligence was only a decade away - a belief shared by many prominent scientists and echoed in generous research funding from DARPA and other agencies.

However, the limitations of single-layer networks were becoming apparent in practice. Many of the problems researchers wanted to solve - pattern recognition, language understanding, game playing - turned out to be far harder than anticipated. The gap between the promise of the perceptron and the reality of what it could achieve was widening.

**1969 - XOR Problem.** Minsky and Papert 1969 published *Perceptrons*, demonstrating formally that single-layer networks could not solve problems as simple as the XOR

function. The 1970s were a difficult period for neural network research. The Lighthill Report Lighthill 1973 delivered a damning assessment of AI research in 1973, concluding that the field had failed to deliver on its promises. DARPA cut funding significantly and many researchers moved away from neural networks entirely. This was the first AI winter.

Yet work continued quietly. Paul Werbos derived the backpropagation algorithm in his 1974 PhD thesis Werbos 1974 - years before it became widely known. Fukushima 1980 introduced the Neocognitron, a hierarchical neural network inspired by the visual cortex that anticipated many ideas later formalised in convolutional networks. Hopfield 1982 networks demonstrated that recurrent networks could function as associative memories. These contributions kept the field alive through the cold years, largely unnoticed by the mainstream AI community which had shifted its focus toward symbolic reasoning and expert systems.

Japan's role in this period deserves particular mention. Fukushima's Neocognitron emerged from NHK Science and Technology Research Laboratories in Tokyo. In 1982 the Japanese government launched the ambitious Fifth Generation Computer Project Ministry of International Trade and Industry 1982, investing heavily in AI hardware and parallel computing. The project caused significant alarm in the US and UK and triggered substantial counter-investment in AI research on both sides of the Atlantic. Its ultimate failure to deliver human-level reasoning contributed to the disillusionment of the second AI winter.

**1986 - Backpropagation.** Rumelhart, G. E. Hinton, and Williams 1986 popularised the backpropagation algorithm, providing a practical method for training multi-layer networks by propagating error gradients backwards through the layers. The revival of backpropagation generated enormous enthusiasm. Multilayer networks were successfully applied to speech recognition, image classification, and financial forecasting. LeCun applied backpropagation to convolutional networks at Bell Labs, demonstrating practical handwriting recognition systems used by the US postal service. Hochreiter and Schmidhuber introduced the Long Short-Term Memory (LSTM) network in 1997 Hochreiter and Schmidhuber 1997, solving the vanishing gradient problem for recurrent networks and laying the foundation for sequence modelling over the following two decades.

However, the second AI winter arrived in the late 1980s and early 1990s. Expert systems - rule-based AI programs that had attracted enormous commercial investment - collapsed as their maintenance costs proved prohibitive and their brittleness became apparent. The Lisp machine market crashed in 1987. DARPA again cut funding. Neural networks hit their own practical ceiling: training deep networks remained unstable, datasets were small, and support vector machines (SVMs) - introduced by Vapnik 1995 - offered better performance on many tasks with stronger theoretical guarantees. By the mid-1990s SVMs had largely displaced neural networks as the method of choice for classification problems.

**1998 - LeNet.** Yann LeCun, Bottou, et al. 1998a demonstrated convolutional neural networks for handwritten digit recognition, achieving state-of-the-art results on the MNIST dataset. Despite its success, neural networks remained out of favour through the early 2000s - a period sometimes called the cold spell - as support vector machines and other kernel methods dominated the field.

This cold spell was not a complete winter - small groups continued working on deep architectures. Bengio, Delalleau, and Le Roux 2004 explored the difficulty of training

deep networks, identifying the vanishing gradient problem as the central obstacle. Hinton's group at Toronto worked on unsupervised pretraining as a way to initialise deep networks before fine-tuning with backpropagation. The theoretical groundwork for the 2006 breakthrough was being laid quietly throughout this period.

**2006 - Deep Belief Nets.** G. E. Hinton, Osindero, and Teh 2006 showed that deep networks could be trained effectively using a layer-wise pretraining procedure . This reignited interest in deep architectures and marked the beginning of the modern deep learning era, even before large datasets and GPU training were fully established.

The key developments of this period were infrastructural rather than algorithmic. Raina, Madhavan, and Ng 2009 demonstrated that GPUs could accelerate neural network training by an order of magnitude, making experiments that previously took weeks feasible in hours. The ImageNet dataset was released by Deng et al. 2009, providing for the first time a benchmark large and challenging enough to reveal the true potential of deep architectures. Nair and G. E. Hinton 2010 introduced the ReLU activation function, which proved dramatically more effective than sigmoid and tanh activations for training deep networks. These three developments - GPU training, large-scale data, and better activations - set the stage for AlexNet's breakthrough in 2012.

**2012 - AlexNet.** Krizhevsky, Sutskever, and G. E. Hinton 2012a entered a deep convolutional network trained on GPUs into the ImageNet challenge and won by a margin that shocked the field - reducing the top-5 error rate from 26% to 15% in a single year. This moment is widely regarded as the turning point that established deep learning as the dominant paradigm in machine learning.

**2017 - Transformers.** Vaswani et al. 2017 introduced the transformer architecture in the paper *Attention Is All You Need*, replacing recurrent computation with self-attention mechanisms. The transformer became the foundation of virtually all modern large-scale models in both language and vision.

**2022 - ChatGPT.** OpenAI released ChatGPT, bringing large language models to a mass audience for the first time. Within two months it had reached 100 million users, making it the fastest-growing consumer application in history and triggering a wave of public and institutional interest in artificial intelligence that continues today.

The pattern visible in Figure 1.2 is worth noting: the history of deep learning is one of recurring cycles of excitement and disappointment, punctuated by genuine breakthroughs. The current era is distinguished not just by the scale of the results but by the breadth of their impact - for the first time, deep learning systems are changing how ordinary people work and communicate on a daily basis.

## 1.4. Why Now?

▹ Session 1: Why Now? - Four Driving Forces

Deep learning is not a new idea - neural networks have existed since the 1950s. What changed is the confluence of four factors that together made the approach viable at scale. None of these alone was sufficient; together they triggered a revolution.

### 1.4.1. Data

▹ Session 1: Why Now? - Four Driving Forces, Data - Scale Revolution

The internet created the largest dataset in history by accident. Every uploaded image, every written caption, every search query became potential training material. This was crystallised deliberately in benchmark datasets - ImageNet (2009) provided 1.2 million labelled images across 1,000 categories, and GLUE provided a standardised benchmark

for natural language understanding. These benchmarks did not just measure progress, they created it by giving the community a shared target to optimise against.

Crucially, the field also learned that labels are not always necessary. Self-supervised and weakly supervised learning techniques extract signal directly from unlabelled data, massively expanding what counts as usable training material. The web, in its entirety, became a dataset.

Figure 1.3 shows the growth in dataset scale over time. Two aspects deserve attention. First, the left axis is logarithmic - each gridline represents a tenfold increase, so the jump from ImageNet to LAION-5B represents roughly four thousand times more data. Second, the orange line shows that resolution grew alongside quantity - from $28 \times 28$ pixels for MNIST to $1024 \times 1024$ for LAION-5B, a 1,000-fold increase in pixels per image. The two dimensions of scale - how many images and how much information per image - grew together.

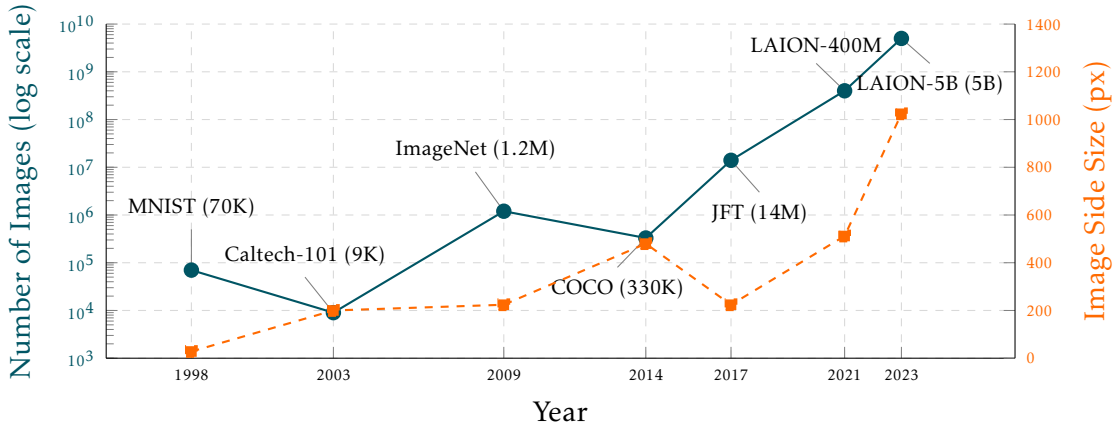


Figure 1.3: Image dataset sizes and resolutions over time. Blue line: number of images (log scale, left axis). Orange line: image side size in pixels (linear scale, right axis).

### 1.4.2. Compute

▹ Session 1: Compute & Progress

Graphics processing units, designed for the parallel computation of gaming graphics, turned out to be ideally suited for the matrix multiplications at the heart of neural network training. NVIDIA's CUDA programming model made GPUs accessible to researchers without hardware expertise. Custom chips - Google's TPUs, Apple Silicon, and a wave of AI-specific inference hardware - followed, with architectures designed around deep learning from the ground up.

Cloud platforms such as AWS, Microsoft Azure, and Google Cloud democratised access to large-scale compute - a researcher can now rent state-of-the-art hardware by the hour. Learning platforms such as Google Colab, Kaggle Notebooks, and Lightning.ai went further, providing free GPU access directly in the browser with zero setup. As Figure 1.4 shows, GPU performance has grown from single-digit TFLOPS in 2012 to nearly 2,000 TFLOPS in 2024.

### 1.4.3. Algorithms

▹ Session 1: Why Now? - Four Driving Forces, Compute & Progress

Several targeted innovations removed practical obstacles that had blocked progress for decades. At the architectural level, the multilayer perceptron (MLP), convolutional neu-

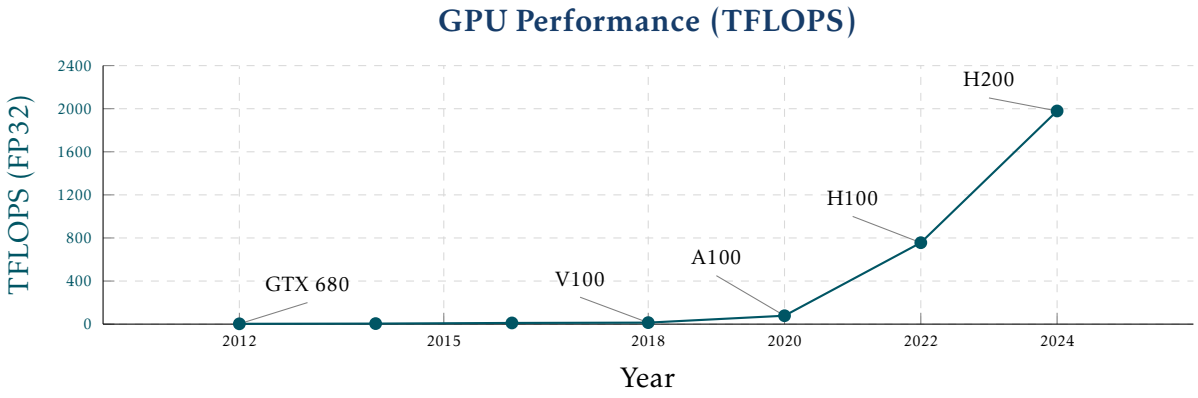


Figure 1.4: GPU performance in TFLOPS (FP32) from 2012 to 2024.

ral network (CNN), and recurrent neural network (RNN) established the foundational building blocks, all trained via the backpropagation algorithm. The ReLU activation function mitigated the vanishing gradient problem that had plagued deep networks. Batch normalisation stabilised training. Dropout provided effective regularisation.

At a higher level, residual connections (ResNets) made it possible to train networks of hundreds of layers. The attention mechanism and the transformer architecture, introduced in 2017, replaced recurrent computation entirely and became the foundation of modern language and vision models.

Figure 1.5 illustrates the impact of these algorithmic advances on the ImageNet benchmark. In 2011, the best classical methods using SVMs achieved around 74% top-5 accuracy. AlexNet's jump to 84.7% in 2012 marked the beginning of the deep learning era. Each subsequent architectural innovation - VGG, ResNet, EfficientNet, ViT - pushed accuracy further, surpassing human performance of approximately 95% around 2015 and reaching over 99% by 2022.

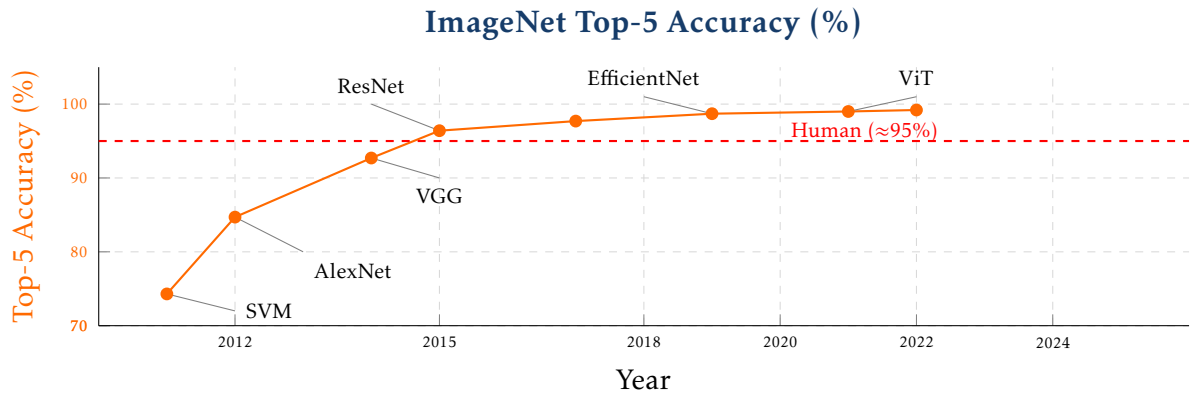


Figure 1.5: ImageNet top-5 accuracy from 2011 to 2022. Each labelled point corresponds to a key architectural innovation. The dashed red line indicates approximate human performance.

### 1.4.4. Community

Perhaps the most underappreciated factor is the open culture of the deep learning community. Frameworks such as PyTorch, TensorFlow, and JAX are open source,

actively maintained, and freely available. The norm of releasing code alongside papers - through platforms such as Papers with Code, Kaggle, and HuggingFace - means that results are reproducible and immediately buildable upon. Research is shared on arXiv before peer review, and major conferences such as NeurIPS, ICML, and ICLR publish proceedings openly.

This compounding effect - where each breakthrough immediately becomes everyone's foundation - has no parallel in most other scientific fields.

## 1.5. What Can Deep Learning Do?

▹ Session 1: What Can Deep Learning Do?

Deep learning today touches almost every domain of human activity. Rather than organising applications by the models used, it is more instructive to group them by the type of task they perform.

### 1.5.1. Language and Communication

Natural language was one of the earliest and remains one of the most impactful application areas. Machine translation systems such as DeepL and Google Translate now produce fluent output across dozens of language pairs, a task that resisted decades of rule-based approaches. Conversational agents - ChatGPT, Claude, Gemini - have brought language models to a mass audience, demonstrating capabilities in reasoning, summarisation, and question answering that were considered out of reach just a few years ago. Code generation tools such as GitHub Copilot and Cursor assist developers by completing, explaining, and generating code in real time.

### 1.5.2. Vision and Perception

Computer vision was the domain in which deep learning first demonstrated decisive superiority over classical methods, with AlexNet's 2012 ImageNet result marking the turning point. Today deep learning powers image classification, object detection, and semantic segmentation across a vast range of applications. Autonomous driving systems parse complex road scenes in real time, combining camera, lidar, and radar inputs to make driving decisions. Satellite and aerial imagery analysis enables large-scale monitoring of land use, deforestation, and infrastructure.

### 1.5.3. Healthcare and Medicine

Medical imaging has been transformed by deep learning. Systems trained on large collections of labelled scans can detect tumours, classify X-rays, and segment anatomical structures with accuracy approaching or exceeding that of specialist clinicians. Beyond imaging, deep learning is applied to genomics - identifying disease-associated variants in DNA sequences - and to clinical decision support systems that assist physicians with diagnosis and treatment planning.

### 1.5.4. Generation

Generative models have emerged as one of the most publicly visible applications of deep learning. Image generation systems such as Stable Diffusion and DALL-E produce photorealistic or stylised images from text descriptions. Video and music

generation - tools such as Suno and Udio - extend this capability to temporal media. The same underlying technology that enables generation also enables detection: deepfake detection systems attempt to identify synthetically generated media, an increasingly important capability as generation quality improves.

### 1.5.5. Scientific Discovery

Perhaps the most consequential application of deep learning to date is AlphaFold Jumper, Evans, Pritzel, et al. 2021, DeepMind's system for predicting protein three-dimensional structure from amino acid sequence. AlphaFold solved a fifty-year-old open problem in structural biology and earned Demis Hassabis and John Jumper the 2024 Nobel Prize in Chemistry - the first Nobel Prize awarded for work driven by deep learning. Deep learning is also applied to drug discovery - predicting molecular properties and identifying candidate compounds - and to climate and weather modelling, where neural networks are beginning to challenge traditional physics-based simulation approaches.

### 1.5.6. Decision Making and Control

Reinforcement learning - a branch of deep learning in which agents learn by interacting with an environment - has produced some of the field's most dramatic results. AlphaGo Silver, Huang, Maddison, et al. 2016 defeated the world champion Go player in 2016, a milestone considered a decade away by most experts at the time. Robotic systems learn manipulation and locomotion skills directly from experience, without hand-coded controllers. Recommendation systems - the algorithms that determine what appears in TikTok feeds, Netflix homepages, and Spotify playlists - are among the most economically impactful applications of deep learning, shaping the daily media consumption of billions of people.

# 2. Convolutional Neural Networks

Convolutional neural networks (CNNs) are a class of neural networks designed to process data that has a natural spatial or sequential structure. Their defining property is the use of a **convolution operation** in place of general matrix multiplication, which endows the network with three structural biases that are indispensable for image processing: **local connectivity**, **parameter sharing**, and **translation invariance**. This session introduces the convolution operation from first principles, builds up to the full CNN architecture, and traces the development from the earliest successful networks to the residual architectures that form the backbone of modern computer vision.

## 2.1. Why Not MLPs for Images?

▸ Images in Computers; Images as Data; How to Look at Images

Before motivating the convolutional layer, it is worth understanding precisely why a standard fully connected network is poorly suited to image data. The problem is not expressiveness — given enough neurons, an MLP can in principle approximate any function — but **computational cost** and **inductive bias**.

### 2.1.1. Images as Structured Tensors

A digital image is represented as a three-dimensional tensor of shape $C \times H \times W$, where $C$ is the number of channels, $H$ is the height in pixels, and $W$ is the width in pixels. Each entry is a pixel intensity, either an integer in the range $[0, 255]$ or a normalised float in $[0, 1]$, stored in 8 bits per channel. The two formats relevant for most deep learning work are **grayscale** ($C = 1$), where pixel values run from black (0) to white (255), and **RGB** ($C = 3$), the standard format for camera images and screens, with separate red, green, and blue channels. Other formats exist – RGBA with an additional transparency channel, CMYK used in printing, and high-dimensional medical or hyperspectral images that may carry many channels at 16-bit depth – but these are typically converted to RGB or grayscale before being fed to a network.

Image sizes vary enormously across tasks. Benchmark datasets designed for classification at modest resolution include MNIST ($28 \times 28$, grayscale, 784 pixels per image) and ImageNet ($224 \times 224$, RGB, 150 528 pixels). Real-world sources produce far larger inputs: a medical CT slice is typically $512 \times 512$ (262 144 pixels), a full-HD screen captures $1920 \times 1080$ pixels (over 6 million), and a modern phone camera can produce $4000 \times 3000$ images with 36 million pixels per frame.

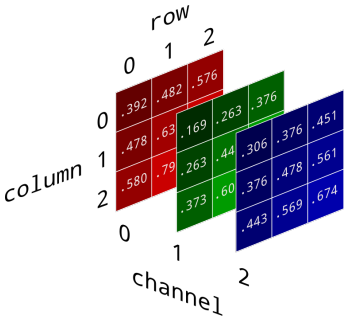


Figure 2.1: An RGB image decomposed into its three colour channels.

### 2.1.2. The Parameter Explosion

The standard MLP approach to images is to **flatten** the input tensor to a vector $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$ and pass it through a sequence of fully connected layers. The weight matrix of the first hidden layer alone has dimensions $(H \cdot W \cdot C) \times d_1$, where $d_1$ is the number of hidden units. For a modest hidden layer of $d_1 = 1024$ units applied to an ImageNet image, this gives $150\,528 \times 1024 \approx 154$ million parameters – for a *single* layer. Training such a model would be prohibitively expensive in memory and computation, and the model would require an enormous amount of data just to avoid overfitting.

### 2.1.3. The Missing Inductive Bias

Parameter count alone does not fully explain the problem. A fully connected layer treats every input pixel as equally related to every other pixel. This is wrong for images: pixels that are spatially close carry far more information about each other than pixels at opposite ends of the image. More importantly, a fully connected layer must **relearn** the same feature detector from scratch at every spatial location. A filter that detects a horizontal edge in the top-left corner of an image is identical in function to one that detects a horizontal edge in the bottom-right, but an MLP represents them as entirely separate parameters.

Human vision operates differently. We look for **local patterns** – edges, corners, textures – and we recognise the same pattern regardless of where it appears in our visual field. Simple detected features combine into increasingly complex ones as we move from early to later visual processing stages. The convolutional layer translates these three properties directly into a neural network primitive: local connectivity

restricts each neuron to a small spatial patch; parameter sharing uses the **same filter weights at every location**; and as a consequence, any feature a filter detects at one position is detected identically everywhere else, giving translation invariance as an emergent property.

## 2.2. The Convolution Operation

▸ Convolution — The Operation; 1D Convolution; 2D Convolution; Padding; Stride; Multiple Input Channels; Multiple Output Channels; Convolution as Matrix Multiplication

The convolution operation is the mathematical engine of a CNN. This section develops it in full generality, starting from the simple one-dimensional case and extending to the multi-channel two-dimensional setting used in practice.

### 2.2.1. 1D Convolution

Let $\mathbf{x} \in \mathbb{R}^n$ be an input signal and $\boldsymbol{\kappa} \in \mathbb{R}^k$ be a **filter** (also called a **kernel**) of length $k$. The 1D convolution of $\mathbf{x}$ with $\boldsymbol{\kappa}$ produces an output signal whose value at position $i$ is the dot product of the filter with the local patch of $\mathbf{x}$ starting at $i$:

$$(\mathbf{x} * \boldsymbol{\kappa})[i] = \sum_{j=0}^{k-1} \mathbf{x}[i+j]\, \boldsymbol{\kappa}[j]. \tag{2.1}$$

Here $i$ indexes the output position and $j$ indexes the position within the filter. The filter **slides** across the signal from left to right: at each position $i$, it computes a single dot product, producing one value of the output. For a filter of length $k = 3$ with weights $(1, -1, 2)$ and the input patch $(1, 3, 4)$ at position $i = 1$, the output is $1 \cdot 1 + 3 \cdot (-1) + 4 \cdot 2 = 6$.

This operation embodies all three desirable properties simultaneously. Local connectivity is enforced because each output value depends on only $k$ contiguous input values, not the entire signal. Parameter sharing is enforced because the same filter weights $\boldsymbol{\kappa}$ are used at every position $i$ – a filter that detects a particular local pattern detects it identically wherever it occurs. Translation invariance follows: if a pattern shifts by one position in the input, the same filter detects it one position later in the output.

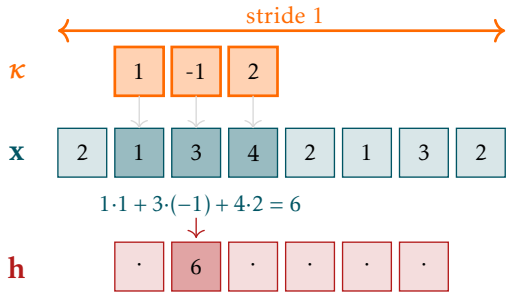


Figure 2.2: 1D convolution: a filter of length 3 slides over an input signal, computing a dot product at each position.

### 2.2.2. 2D Convolution

For images, the filter must slide in two spatial dimensions. Given a single-channel input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and a filter $\boldsymbol{\kappa} \in \mathbb{R}^{k \times k}$, the 2D convolution at output position $(i, j)$ is:

$$(\mathbf{X} * \boldsymbol{\kappa})[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[i+p, j+q]\, \boldsymbol{\kappa}[p, q]. \tag{2.2}$$

The double sum ranges over all rows $p$ and columns $q$ within the filter. Output position $(i, j)$ thus sees the $k \times k$ patch of $\mathbf{X}$ anchored at $(i, j)$. Without any padding, the filter can be placed at $(H-k+1)\times(W-k+1)$ valid positions, so the output has shape $(H-k+1)\times(W-k+1)$. For a $5 \times 5$ input with a $3 \times 3$ filter, the output is $3 \times 3$.

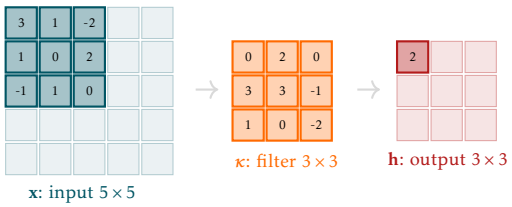


Figure 2.3: 2D convolution: a $3 \times 3$ filter applied to a single spatial location of a $5 \times 5$ input. The filter computes a dot product with the local patch.

### 2.2.3. Padding

A practical difficulty with the basic formula above is that the output is smaller than the input. With $L$ convolutional layers and no padding, the spatial dimensions shrink by $k-1$ at each layer, quickly consuming the available resolution. For a 10-layer network with $3 \times 3$ filters, a $224 \times 224$ input would shrink to $204 \times 204$, which may be acceptable, but a $28 \times 28$ input would be exhausted long before the network is deep enough to learn useful features.

The standard solution is to add a border of zeros around the input before convolving. Two conventions are in common use. **Valid padding** ($p = 0$) applies no padding at all, so the output strictly shrinks; this is appropriate when spatial reduction at each layer is intentional. **Same padding** sets $p = (k-1)/2$, which adds enough zeros to keep the output the same size as the input for odd filter sizes. Same padding is by far the most common choice in modern architectures: it decouples spatial resolution from filter size and makes it straightforward to design networks of arbitrary depth. Border pixels receive less coverage than central pixels, but in practice this edge effect is negligible.

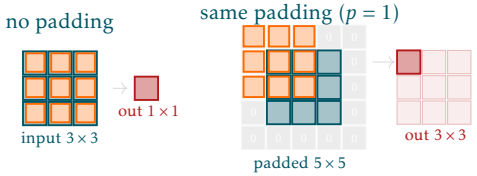


Figure 2.4: Same padding adds zeros around the input border so the output has the same spatial dimensions as the input.

### 2.2.4. Stride

Stride controls **how far the filter moves** between consecutive applications. With stride $s = 1$ (the default), the filter advances one pixel at a time. With stride $s = 2$, it skips every other position, halving the output dimensions. The general output size formula for an input of height $H$, filter size $k$, padding $p$, and stride $s$ is:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \tag{2.3}$$

and analogously for the width. Strided convolution is an alternative to pooling for spatial downsampling: it reduces resolution while simultaneously applying a learned filter, rather than discarding information with a fixed max or average operation. Modern architectures such as ResNet use strided convolution at resolution-changing boundaries instead of pooling.

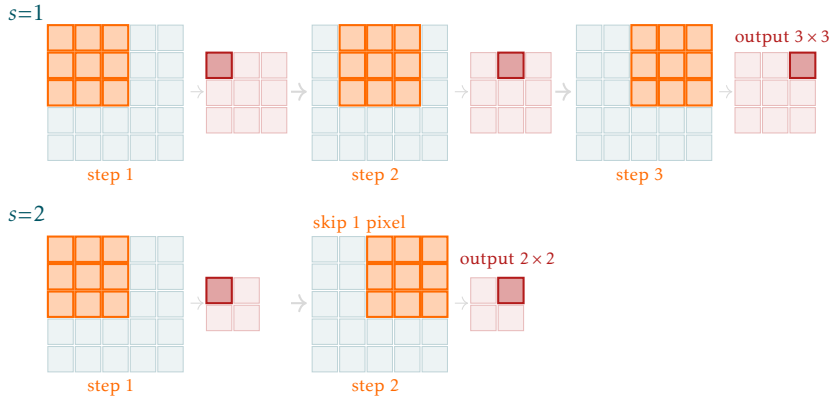


Figure 2.5: Stride 1 (top) and stride 2 (bottom) applied to the same input. Stride 2 halves the output spatial dimensions.

### 2.2.5. Multiple Input Channels

Real images have $C_{\text{in}} > 1$ channels. A single filter must respond to the full depth of its input, so it too must have $C_{\text{in}}$ slices – one per input channel. The filter $\boldsymbol{\kappa} \in \mathbb{R}^{C_{\text{in}} \times k \times k}$ is applied by convolving each slice independently with its corresponding input channel and summing the results:

$$(\mathbf{X} * \boldsymbol{\kappa})[i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q]\, \boldsymbol{\kappa}[c, p, q]. \tag{2.4}$$

The inner double sum convolves filter slice $c$ with input channel $c$; the outer sum over $c$ accumulates the results across all channels. Regardless of $C_{\text{in}}$, a single filter produces a single **feature map** – a $1 \times H' \times W'$ output. The feature map captures a single learned pattern detected across all input channels simultaneously.

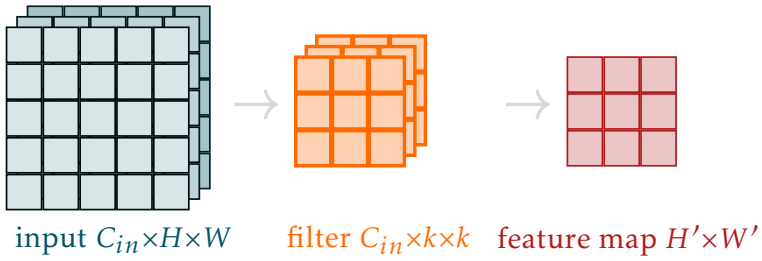


Figure 2.6: A filter with $C_{\text{in}}$ slices applied to a multi-channel input. Each filter slice convolves with one channel; the results are summed to produce one feature map.

### 2.2.6. Multiple Output Channels

A single filter detects a single pattern. To detect $C_{\text{out}}$ different patterns, we use $C_{\text{out}}$ independent filters, each producing its own feature map. Filter $f$ produces:

$$\mathbf{h}_f[i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q]\, \boldsymbol{\kappa}_f[c, p, q], \qquad f = 0, \ldots, C_{\text{out}} - 1. \tag{2.5}$$

The $C_{\text{out}}$ feature maps are stacked into an output tensor of shape $C_{\text{out}} \times H' \times W'$. The total number of learnable parameters in the layer, including one bias per filter, is $C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}}$. This is **independent of the spatial dimensions** $H$ and $W$ of the input – a critical contrast with a fully connected layer, where parameter count grows linearly with image size.

In typical CNN designs, the number of channels **increases** with depth while spatial dimensions decrease. Early layers use few filters to detect simple low-level patterns in the original image resolution; later layers use many filters to detect complex semantic patterns in reduced-resolution feature maps.

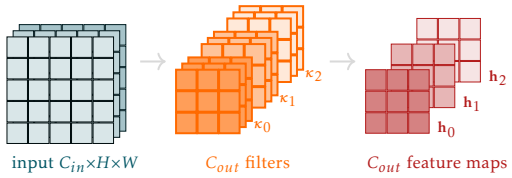


Figure 2.7: $C_{\text{out}}$ filters each applied independently to the full input volume, producing $C_{\text{out}}$ feature maps stacked into the output tensor.

### 2.2.7. Convolution as Sparse, Tied Matrix Multiplication

It is instructive to observe that convolution is **a special case of matrix multiplication**. If we flatten both the input and the output to vectors, the convolution operation can be written as $\mathbf{h} = \mathbf{W}\mathbf{x}$ where $\mathbf{W}$ is a specific matrix structure. Two structural constraints distinguish it from a general weight matrix. First, **sparsity**: each row of $\mathbf{W}$ has only $k^2$ non-zero entries (the filter size), because each output neuron depends on only a small local patch of the input. Second, **weight tying**: the non-zero entries in every row are identical copies of the same filter weights, shifted by one position from row to row.

This perspective is theoretically important for two reasons. Since convolution is a linear transformation, the composition Conv → ReLU → Conv → ReLU → ⋯ has exactly the same representational structure as an MLP, but with far fewer parameters and an explicit inductive bias toward spatial locality. Furthermore, the sparse tied structure means that the gradient of the loss with respect to the filter weights is simply the **sum of gradients across all spatial positions** – which is exactly the correlation operation applied in backpropagation.

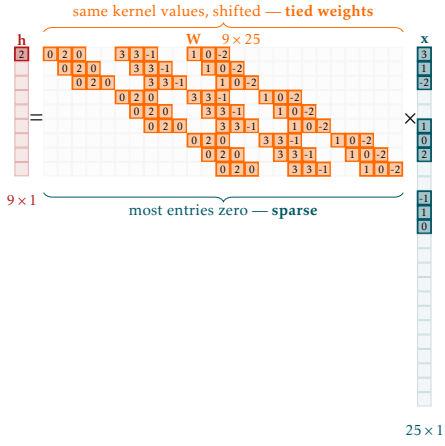


Figure 2.8: The weight matrix corresponding to a convolution. Non-zero entries (coloured) repeat the same filter values; most entries are zero (local connectivity).

## 2.3. CNN Architecture

Individual convolutional layers are stacked into a complete network according to a design pattern that has remained largely stable since the early 2010s. This section describes how the layers fit together and what the resulting network learns.

▹ Receptive Field; What Do CNNs Learn?; Pooling; CNN Building Block; Full CNN Architecture

### 2.3.1. Receptive Field

The **receptive field** of a neuron is the region of the input image that can influence that neuron's

output. For a single convolutional layer with a $3 \times 3$ filter, each output neuron sees a $3 \times 3$ patch of the input. After a second $3 \times 3$ layer, each output neuron is influenced by a $5 \times 5$ patch of the *first* layer's input, because its $3 \times 3$ neighbourhood in the first feature map each covers a $3 \times 3$ patch of the original input. In general, $L$ layers of $3 \times 3$ convolutions yield a receptive field of $(2L+1) \times (2L+1)$.

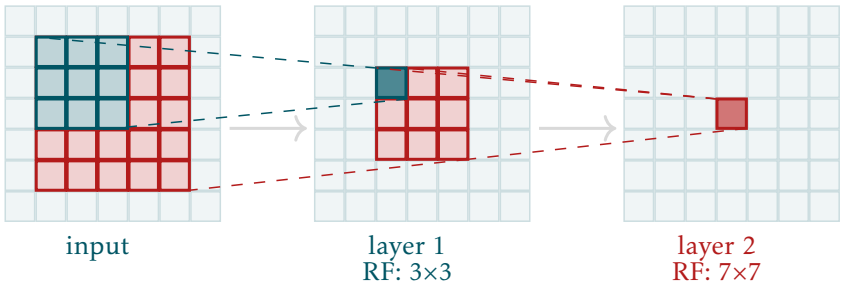


Figure 2.9: Receptive field growth with depth. Each additional $3 \times 3$ convolutional layer extends the receptive field by 2 pixels in each dimension.

This is the key to the **hierarchical feature** story. Early layers have small receptive fields and detect simple local patterns: oriented edges, colour gradients, and small textures. Later layers, with large receptive fields, can combine these local detections into representations of complex structures such as object parts or entire objects. Zeiler and Fergus (2014) made this hierarchy directly visible by developing a technique for projecting feature map activations back to pixel space. Their results confirmed that early layers respond to edges and colour blobs, intermediate layers to textures and repeated patterns, and deep layers to object parts and faces.

### 2.3.2. Pooling

Convolution produces feature maps that are spatially highly redundant: neighbouring output neurons have strongly overlapping receptive fields and therefore highly correlated values. **Pooling** is a fixed, non-trainable operation that reduces spatial redundancy by summarising each local region into a single value.

The most common variant is **max pooling**, which takes the maximum value within a rectangular region $\mathcal{R}_{ij}$:

$$y[i,j] = \max_{p,q \in \mathcal{R}_{ij}} x[p,q]. \tag{2.6}$$

Max pooling retains the strongest activation in each region, which corresponds intuitively to detecting whether a feature is present at all within that neighbourhood, regardless of its precise location. **Average pooling** instead computes the mean, which carries information about the average activation level. The pooling operation applies independently to each channel, leaving the channel dimension unchanged.

A $2 \times 2$ pool with stride 2 (the standard choice) halves both spatial dimensions: a $C \times H \times W$ feature map becomes $C \times H/2 \times W/2$. Each pooling layer therefore doubles the effective receptive field of all subsequent neurons. Pooling

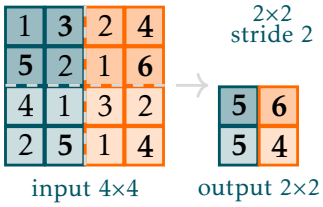


Figure 2.10: 2×2 max pooling reduces each $2 \times 2$ region to its maximum, halving both spatial dimensions.

and strided convolution serve a similar purpose – downsampling – but strided convolution is generally preferred in modern architectures because the downsampling filter is learned rather than fixed.

### 2.3.3. The Standard CNN Building Block

Modern CNNs organise their convolutional layers into a repeating **building block**. The standard pattern is:

$$\text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow (\text{Pool}),$$

where the pooling step is applied selectively rather than after every layer. Each component has a distinct role: the convolution extracts features via learned filters; batch normalisation stabilises the distribution of activations and is essential for training networks deeper than roughly ten layers; ReLU introduces the non-linearity without which a stack of convolutional layers collapses to a single linear map; and pooling reduces spatial resolution.

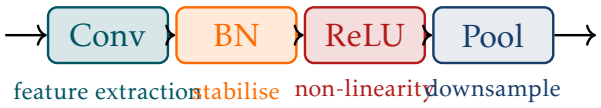


Figure 2.11: The standard CNN building block: Conv → BatchNorm → ReLU, with optional pooling for spatial downsampling.

### 2.3.4. Full CNN Architecture: Backbone and Head

A complete CNN for image classification consists of two functionally distinct stages. The first stage – the **backbone** – is a sequence of convolutional blocks that transform the raw image into a rich feature representation. Across these blocks, the spatial dimensions decrease (via pooling or strided convolution) while the channel count increases, following a characteristic pattern such as $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$ channels. The backbone acts as a **general-purpose feature extractor**: its filters learn to detect progressively more abstract and semantically meaningful patterns in the image.

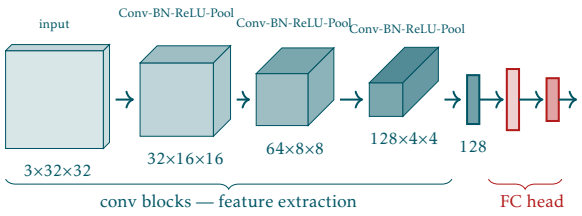


Figure 2.12: Full CNN architecture for image classification. The backbone (convolutional blocks) extracts features; the head (fully connected layers) maps to class scores.

The second stage – the **head** – maps the extracted feature representation to the desired output. For image classification, the head typically flattens (or globally average-pools) the final feature map and passes it through one or more fully connected layers, ending with a softmax over the class scores. The head is inherently **task-specific**: the same backbone can be paired with different heads for classification, detection, or segmentation. This backbone/head distinction becomes central in Session 10, when we discuss how to reuse pretrained backbones for new tasks.

## 2.4. Classic Architectures

▹ LeNet-5; AlexNet; VGG; ResNet; Residual Block

The history of CNN architectures is not merely a catalogue of engineering milestones – it is a record of the key ideas that unlocked the potential of deep learning for vision. Each architecture discussed below contributed one or more insights that are now standard practice.

### 2.4.1. LeNet-5

**LeNet-5** (LeCun, Bottou, et al. 1998b) was the first CNN to demonstrate practical success on a real-world task. Trained on handwritten digits, it introduced the now-canonical pattern of alternating convolutional layers and pooling layers followed by fully connected classification layers:

$$\text{Conv} \to \text{Pool} \to \text{Conv} \to \text{Pool} \to \text{FC} \to \text{FC},$$

with approximately 60 000 parameters in total. The network was small by modern standards but sufficiently powerful to be deployed commercially for cheque reading. Its fundamental limitation was a shortage of compute: deeper networks capable of handling more complex tasks could not be trained efficiently on the hardware of the 1990s.

### 2.4.2. AlexNet

The publication of **AlexNet** by Krizhevsky, Sutskever, and G. E. Hinton (2012b) in 2012 is generally identified as the beginning of the modern deep learning era for computer vision. AlexNet won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2012) by a margin that stunned the computer vision community: it achieved a top-5 error rate of 15.3%, compared to 26.2% for the second-placed entry. The architecture – five convolutional layers followed by three fully connected layers, with approximately 60 million parameters – was not radically different in structure from LeNet, but it incorporated four innovations that collectively made deep CNNs tractable. **ReLU activations** replaced sigmoid and tanh nonlinearities, dramatically accelerating training by avoiding the vanishing gradient problem that afflicts saturating activations. **Dropout** was applied in the fully connected layers as a regulariser. **Data augmentation** – random crops and horizontal flips during training – reduced overfitting. And critically, the network was trained on two **NVIDIA GTX 580 GPUs**, demonstrating that GPU parallelism could make large-scale CNN training feasible.

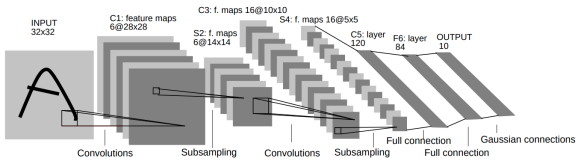


Figure 2.13: LeNet-5 architecture: two convolutional blocks followed by fully connected layers for digit classification.

### 2.4.3. VGGNet

**VGGNet** (Simonyan and Zisserman 2015) pursued a different question: what is the effect of network depth when all other design choices are held constant? The answer was a principled simplification: use only $3 \times 3$ filters throughout the entire network,

and increase depth systematically. The flagship VGG-16 model uses 13 convolutional layers and 3 fully connected layers with approximately 138 million parameters.

The key insight was that two stacked $3 \times 3$ convolutions cover the same receptive field as one $5 \times 5$ convolution, but with strictly fewer parameters ($2 \times 9 = 18$ vs 25) and an additional non-linearity between them. This makes the network more expressive while remaining parameter-efficient. The main weakness of VGGNet is its three large fully connected layers, which together account for roughly 100 million of the 138 million parameters and impose a heavy memory burden. Despite this, VGGNet's clean and uniform design made it an excellent baseline and its features were widely reused for transfer learning.

| ConvNet Configuration | | | | | |
|---|---|---|---|---|---|
| A | A-LRN | B | C | D | E |
| 11 weight layers | 11 weight layers | 13 weight layers | 16 weight layers | 16 weight layers | 19 weight layers |
| input (224 × 224 RGB image) | | | | | |
| conv3-64 | conv3-64 **LRN** | conv3-64 **conv3-64** | conv3-64 conv3-64 | conv3-64 conv3-64 | conv3-64 conv3-64 |
| maxpool | | | | | |
| conv3-128 | conv3-128 | conv3-128 **conv3-128** | conv3-128 conv3-128 | conv3-128 conv3-128 | conv3-128 conv3-128 |
| maxpool | | | | | |
| conv3-256 conv3-256 | conv3-256 conv3-256 | conv3-256 conv3-256 | conv3-256 conv3-256 **conv1-256** | conv3-256 conv3-256 **conv3-256** | conv3-256 conv3-256 conv3-256 **conv3-256** |
| maxpool | | | | | |
| conv3-512 conv3-512 | conv3-512 conv3-512 | conv3-512 conv3-512 | conv3-512 conv3-512 **conv1-512** | conv3-512 conv3-512 **conv3-512** | conv3-512 conv3-512 conv3-512 **conv3-512** |
| maxpool | | | | | |
| conv3-512 conv3-512 | conv3-512 conv3-512 | conv3-512 conv3-512 | conv3-512 conv3-512 **conv1-512** | conv3-512 conv3-512 **conv3-512** | conv3-512 conv3-512 conv3-512 **conv3-512** |
| maxpool | | | | | |
| FC-4096 | | | | | |
| FC-4096 | | | | | |
| FC-1000 | | | | | |
| soft-max | | | | | |

Figure 2.14: VGG architecture: uniform $3 \times 3$ filters stacked to increasing depth.

### 2.4.4. ResNet

By 2015, it was well established that deeper networks generally achieve better accuracy, yet practitioners had observed a frustrating phenomenon: beyond a certain depth, adding more layers actually **degraded** training accuracy. This was not overfitting – the training loss itself was higher for deeper models – but an optimisation failure. **ResNet** (He et al. 2016) diagnosed the problem and proposed a structural fix.

The key observation is that a deeper network should, in principle, be at least as good as its shallower counterpart: the additional layers could simply learn the identity mapping and pass their input through unchanged. Yet learning the identity is apparently difficult for a standard convolutional layer. The **residual connection** (also called a skip connection) reformulates the problem by adding a direct shortcut from the input of a block to its output:

$$\mathbf{y} = \mathrm{ReLU}(\mathcal{F}(\mathbf{x}) + \mathbf{x}). \tag{2.7}$$

Here $\mathcal{F}(\mathbf{x})$ represents two (or three) convolutional layers. Instead of learning the full output mapping $\mathbf{y} = \mathcal{F}(\mathbf{x})$, the block learns the **residual** $\mathcal{F}(\mathbf{x}) = \mathbf{y} - \mathbf{x}$. When the optimal transformation is close to the identity, the block only needs to learn values close to zero rather than memorising a copy of its input – a much easier optimisation task. The shortcut also provides a **direct gradient path** from the loss back to early layers, substantially reducing vanishing gradient problems in very deep networks.

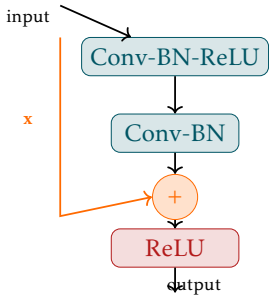


Figure 2.15: Residual block: the block learns the residual $\mathcal{F}(\mathbf{x}) = \mathbf{y} - \mathbf{x}$ rather than the full mapping $\mathbf{y}$.

The basic residual block consists of two Conv-BN-ReLU sequences. For deeper variants (ResNet-50 and above), a **bottleneck block** replaces these two layers with three: a $1 \times 1$ convolution that reduces the channel count, a $3 \times 3$ convolution that processes the reduced representation, and a $1 \times 1$ convolution that restores the channel count. This achieves a similar receptive field to the basic block with significantly fewer parameters. When the shortcut must bridge layers with different channel

counts or spatial dimensions, a **projection shortcut** is used: a $1 \times 1$ strided convolution applied to $\mathbf{x}$ to match the shape of $\mathcal{F}(\mathbf{x})$.

With residual connections, He et al. (2016) successfully trained networks with 50, 101, and 152 layers – depths that had been completely inaccessible before. ResNet remains one of the most widely used backbone architectures in computer vision.

## 2.5. Modern Details

▸ 1 × 1 Convolutions; Batch Norm in CNNs; Global Average Pooling

Beyond the high-level architecture choices discussed above, several lower-level design elements have become standard ingredients in modern CNNs. This section covers three of the most consequential: $1 \times 1$ convolutions, batch normalisation adapted to the convolutional setting, and global average pooling.

### 2.5.1. $1 \times 1$ Convolutions

A $1 \times 1$ convolution applies a learned linear combination across channels at each spatial location independently:

$$y_c[i, j] = \sum_{c'=0}^{C_{\text{in}}-1} w_{c,c'} \, x_{c'}[i, j] + b_c. \tag{2.8}$$

This is equivalent to applying a fully connected layer at every spatial position with shared weights. Despite its name, a $1 \times 1$ convolution has no spatial extent: it performs no spatial mixing, but it mixes information **across channels**. Its uses are widespread: it changes the channel count cheaply (expanding or compressing the feature volume), injects a non-linearity between channel combinations without spatial processing, and serves as the projection shortcut in residual connections when dimensions must change.

### 2.5.2. Batch Normalisation in CNNs

Batch normalisation was introduced in Session 7 for fully connected networks. In the convolutional setting, the natural generalisation is **BatchNorm2d**, which normalises over the batch *and* spatial dimensions for each channel separately. For channel $c$, the mean and variance are computed as:

$$\mu_c = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} x_c^{(n)}[i, j], \tag{2.9}$$

$$\sigma_c^2 = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} \left( x_c^{(n)}[i, j] - \mu_c \right)^2, \tag{2.10}$$

where $N$ is the batch size. There is one mean and variance per channel – not per spatial location – reflecting the parameter-sharing philosophy of convolution: the statistics of a feature map should be consistent across spatial positions. Each channel also has learnable scale and shift parameters $\gamma_c$ and $\beta_c$.

The standard placement in a CNN block is Conv → BatchNorm2d → ReLU. BatchNorm2d is practically essential for training deep CNNs: without it, networks deeper than roughly ten layers become extremely difficult to optimise. It allows higher learning rates, reduces sensitivity to weight initialisation, and acts as a mild regulariser.

### 2.5.3. Global Average Pooling

Flattening the final convolutional feature map before passing it to fully connected layers can be expensive. If the last feature map has shape $C \times H' \times W'$, the flattened vector has $C \cdot H' \cdot W'$ entries, and a fully connected layer from that vector to $d$ hidden units requires $C \cdot H' \cdot W' \cdot d$ parameters.

**Global average pooling** (GAP) eliminates this cost by reducing each feature map to a single scalar:

$$y_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_c[i, j], \qquad c = 1, \ldots, C. \tag{2.11}$$

The result is a vector of length $C$ – one number per channel – which is then passed directly to the classification layer without any intermediate fully connected layers. The intuition is that each feature map represents one learned concept; its average activation across the image measures how strongly that concept is present. This representation is compact, spatial-dimension-agnostic (the same GAP layer works for any input resolution), and dramatically reduces parameter count compared to flattening. In PyTorch, GAP is implemented as `nn.AdaptiveAvgPool2d((1, 1))` followed by `flatten`.

## 2.6. Summary

Convolutional neural networks impose three structural biases on the processing of spatial data: local connectivity, parameter sharing, and translation invariance. Together, these allow CNNs to extract meaningful features from images using orders of magnitude fewer parameters than a comparably capable fully connected network. The building blocks – convolution, batch normalisation, ReLU, and pooling – are assembled into a backbone that extracts a hierarchy of increasingly abstract features, followed by a task-specific head.

The development from LeNet (1998) to AlexNet (2012), VGGNet (2014), and ResNet (2016) charts the empirical discovery of the principles that make deep CNNs work in practice: GPU training, ReLU activations, systematic use of depth, and – most consequentially – residual connections that allow gradient information to flow through networks of arbitrary depth. Session 10 builds directly on the backbone/head distinction introduced here, showing how pretrained backbones can be reused for new tasks without training from scratch.

## Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.

## References


Bengio, Yoshua, Olivier Delalleau, and Nicolas Le Roux (2004). "The Curse of Highly Variable Functions for Local Kernel Machines". In: *Advances in Neural Information Processing Systems*.

Deng, Jia et al. (2009). "ImageNet: A Large-Scale Hierarchical Image Database". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Fukushima, Kunihiko (1980). "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position". In: *Biological Cybernetics* 36, pp. 193–202.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Available at https://www.deeplearningbook.org. MIT Press.

He, Kaiming et al. (2016). "Deep Residual Learning for Image Recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. URL: https://arxiv.org/abs/1512.03385.

Hinton, Geoffrey E., Simon Osindero, and Yee-Whye Teh (2006). "A Fast Learning Algorithm for Deep Belief Nets". In: *Neural Computation* 18.7, pp. 1527–1554.

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.

Hopfield, John J. (1982). "Neural Networks and Physical Systems with Emergent Collective Computational Abilities". In: *Proceedings of the National Academy of Sciences* 79.8, pp. 2554–2558.

Jumper, John, Richard Evans, Alexander Pritzel, et al. (2021). "Highly Accurate Protein Structure Prediction with AlphaFold". In: *Nature* 596, pp. 583–589.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton (2012a). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25.

– (2012b). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. URL: https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep Learning". In: *Nature* 521, pp. 436–444.

LeCun, Yann, Léon Bottou, et al. (1998a). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

– (1998b). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324. DOI: 10.1109/5.726791.

Lighthill, James (1973). *Artificial Intelligence: A General Survey*. Tech. rep. Available at https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf. Science Research Council.

Ministry of International Trade and Industry (1982). *Outline of Research and Development Plans for Fifth Generation Computer Systems*. Tech. rep. Institute for New Generation Computer Technology.

Minsky, Marvin and Seymour Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Nair, Vinod and Geoffrey E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the International Conference on Machine Learning*.

Raina, Rajat, Anand Madhavan, and Andrew Y. Ng (2009). "Large-Scale Deep Unsupervised Learning Using Graphics Processors". In: *Proceedings of the International Conference on Machine Learning*.

Rosenblatt, Frank (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6, pp. 386–408.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams (1986). "Learning Representations by Back-propagating Errors". In: *Nature* 323, pp. 533–536.

Silver, David, Aja Huang, Chris J. Maddison, et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529, pp. 484–489.

Simonyan, Karen and Andrew Zisserman (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1409.1556.

Vapnik, Vladimir (1995). *The Nature of Statistical Learning Theory*. Springer.

Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30.

Werbos, Paul (1974). "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences". PhD thesis. Harvard University.

Widrow, Bernard and Marcian E. Hoff (1960). "Adaptive Switching Circuits". In: *IRE WESCON Convention Record*. Vol. 4, pp. 96–104.

Zeiler, Matthew D. and Rob Fergus (2014). "Visualizing and Understanding Convolutional Networks". In: *European Conference on Computer Vision*, pp. 818–833. URL: https://arxiv.org/abs/1311.2901.